

DRAFT C++ Synchronized Buffered Ostream

ISO/IEC JTC1 SC22 WG21 d0053r4 - 2017-03-02

Lawrence Crowl, Lawrence@Crowl.org

Peter Sommerlad, Peter.Sommerlad@hsr.ch

Nicolai Josuttis

Pablo Halpern

[Introduction](#)

[Solution](#)

[Design](#)

[Feature Test](#)

[Wording](#)

[Headers \[headers\]](#)

[General \[input.output.general\]](#)

[Forward declarations \[iostream.forward\]](#)

[Synchronized output stream \[syncstream\]](#)

[Overview \[syncstream.overview\]](#)

[Class template basic_syncbuf \[syncstream.syncbuf\]](#)

[basic_syncbuf constructors \[syncstream.syncbuf.ctor\]](#)

[basic_syncbuf destructor \[syncstream.syncbuf.dtor\]](#)

[basic_syncbuf assign and swap \[syncstream.syncbuf.assign\]](#)

[basic_syncbuf member functions \[syncstream.syncbuf.mfun\]](#)

[basic_syncbuf overridden virtual functions \[syncstream.syncbuf.virtuals\]](#)

[Class template basic_osyncstream \[syncstream.osyncstream\]](#)

[osyncstream Constructor \[syncstream.osyncstream.ctor\]](#)

[osyncstream Destructor \[syncstream.osyncstream.dtor\]](#)

[osyncstream Assignment \[syncstream.osyncstream.assign\]](#)

[osyncstream Member Functions \[syncstream.osyncstream.mfun\]](#)

[Implementation](#)

[Revisions](#)

[References](#)

Introduction

At present, some stream output operations guarantee that they will not produce race conditions, but do not guarantee that the effect will be sensible. Some form of external synchronization is required. Unfortunately, without a standard mechanism for synchronizing, independently developed software will be unable to synchronize.

[N3535 C++ Stream Mutexes](#) proposed a standard mechanism for finding and sharing a mutex on streams. At the Spring 2013 standards meeting, the Concurrency Study Group requested a change away from a full mutex definition to a definition that also enabled buffering.

[N3678 C++ Stream Guards](#) proposed a standard mechanism for batching operations on a stream. That batching may be implemented as mutexes, as buffering, or some combination of both. It was the response to the Concurrency Study Group. A draft of that paper was reviewed in the Library Working Group, who found too many open issues on what was reasonably exposed to the 'buffering' part.

[N3665 Uninterleaved String Output Streaming](#) proposed making streaming of strings of length less than `BUFSIZ` appear uninterleaved in the output. It was a "minimal" functionality change to the existing standard to address the problem. The full Committee chose not to adopt that minimal solution.

[N3750 C++ Ostream Buffers](#) proposed an explicit buffering, at the direction of the general consensus in the July 2013 meeting of the Concurrency Study Group. In November 2014 a further updated version [N4187](#) was discussed in Library Evolution in Urbana and it was consensus to work with a library expert to get the naming and wording better suited to LWG expectations. Nico Josuttis volunteered to help the original authors. More information on the discussion is available at [LEWG wiki](#) and the corresponding [LEWG bug tracker entry \(20\)](#). This paper address issues raised with [N4187](#). This paper has a change of title to reflect a change in class name, but contains the same fundamental approach.

Solution

We propose a `basic_osyncstream`, that buffers output operations for a wrapped stream. The `basic_osyncstream` will atomically transfer the contents of an internal stream buffer to a `basic_ostream`'s stream buffer on destruction of the `basic_osyncstream`.

The transfer on destruction simplifies the code and ensures at least some output in the presence of an exception.

The intent is that the `basic_osyncstream` is an automatic-duration variable with a relatively small scope which constructs the text to appear uninterleaved. For example,

```
{
    osyncstream bout(cout);
    bout << "Hello, " << "World!" << '\n';
}
```

Design

Here we list a set of objection to our proposal in the form of questions. We then give our reasons why other potential solutions do not work as well as our proposed solution.

Why can I not just use `cout`? It should be thread-safe.

You will not get data races for `cout`, but that is not true for most other streams. In addition there is no guarantee that output from different threads appears in any sensible order. Coherent output order is the main reason for this proposal.

Why must `osyncstream` be an `ostream`? Could a simple proxy wrapper work?

To support all existing and user-defined output operators, `osyncstream` must be an `ostream` subclass.

Can you make a flush of the `osyncstream` mean transfer the characters and flush the underlying stream?

No, because the point of the `osyncstream` is to collect text into an atomic unit. Flushes are often emitted in calls where the body is not visible, and hence unintentionally break up the text. Furthermore, there may be more than one flush within the lifetime of an `osyncstream`, which would impose a performance loss when an atomic unit of text needs only one flush. The design decision is only to flush the underlying stream, if the `osyncstream` was flushed, and only once per atomic transfer of the character sequence.

Can you flush just transfer the characters and not flush the underlying stream?

The flush intends an effect on visible to the user. So, we should preserve at least one flush. The flush may not be visible to the code around the `osyncstream`, and so its programmer cannot do a manual flush, because attempting to flush the underlying stream that is shared among threads will introduce a data race.

Why do you specify `basic_syncbuf`? LWG and LEWG thought you wouldn't need it.

Users use the `streambuf` interface. Without access to the `basic_syncbuf` they would not be able call `emit()` on the underlying `streambuf` responsible the synchronization. Ask Pablo Halpern if you need more to be convinced.

Where will the required lock/mutex be put? Will it be in every `streambuf` object changing the ABI?

That is one of the reasons, why this must be put into the standard library. It is possible to implement with a global map from `streambuf*` to mutexes as the example code does, however, existing standard library implementations might have already a space for the mutexes (not breaking their ABI), because `cout/cerr` seem to require one and those might be the most important ones to wrap.

While it has been pointed out that the implementation should actually reside in the used stream buffer object, we believe the specification should hide that effect, because such a stream buffer is not intended to be ever be used standalone. Therefore, the specification here only provides the `basic_osyncstream` class template and not any details on the underlying stream buffer implementing the mechanics. We have an implementation, where the `basic_osyncstream` is more or less just a shell to the real meat in the stream buffer but that stream buffer is not meant to be exchanged or shared or anything.

We follow typical stream conventions of `basic_` prefixes and typedefs.

The constructor for `osyncstream` takes a non-const reference to a `basic_ostream` obtaining its stream buffer or a `basic_streambuf`. This stream buffer indicates that the destruction of the `osyncstream` may write to the stream buffer obtained with the constructor argument.

The wording below permits implementation of `basic_osyncstream` with either a `stream_mutex` from [N3535](#) or with implementations

suitable for [N3665](#), e.g. with Posix file locks [\[PSL\]](#)

Feature Test

No header called `<syncstream>` exists; testing for this header's existence is thus sufficient for testing existence of the feature.

Wording

This wording is relative to the current C++(17) working draft, but doesn't use section numbers. This allows its integration into the Concurrency TS 2 accordingly.

Headers [headers]

Add a new entry to table — C++ library headers :

<syncstream>

General [input.output.general]

Add a new row to table — Input/output library summary.

xx.12 Synchronized output streams <syncstream>

Forward declarations [iostream.forward]

Add the following forward declarations to the synopsis of `<iosfwd>` in the namespace `std`.

```
template <class charT,
          class traits = char_traits<charT>.,
          class Allocator = allocator<charT>>
    class basic_syncbuf;
using syncbuf = basic_syncbuf<char>;
using wsyncbuf = basic_syncbuf<wchar_t>;

template <class charT,
          class traits = char_traits<charT>.,
          class Allocator = allocator<charT> >
    class basic_ostream;
typedef basic_ostream<char> ostream;
typedef basic_ostream<wchar_t> wostream;
```

Synchronized output stream [syncstream]

Add a new section with the following subsections.

Overview [syncstream.overview]

The header `<syncstream>` provides a mechanism to synchronize execution agents writing to the same stream. It defines a class template `basic_ostream` to buffer output and transfer the buffered content into an object of type `basic_streambuf<charT,traits>` atomically with respect to such transfers by other `basic_ostream<charT,traits,Allocator>` objects referring to the same `basic_streambuf<charT,traits>` object. The transfer occurs when `emit()` is called and when the `basic_ostream<charT,traits,Allocator>` object is destroyed.

Add a synopsis for header `<syncstream>`.

```
template <class charT,
          class traits = char_traits<charT>.,
          class Allocator = allocator<charT>>
    class basic_syncbuf;
```

```
template <class charT,
          class traits,
          class Allocator>
class basic_ostream;
```

Class template basic_syncbuf [syncstream.syncbuf]

```
template <class charT,
          class traits = char_traits<charT>,
          class Allocator = allocator<charT>>
class basic_syncbuf
: public std::basic_streambuf<charT, traits, Allocator> {

public:
    using char_type      = charT;
    using int_type       = typename traits::int_type;
    using pos_type       = typename traits::pos_type;
    using off_type       = typename traits::off_type;
    using traits_type     = traits;
    using allocator_type = Allocator;

    using streambuf_type = basic_streambuf<charT, traits>;

    explicit
    basic_syncbuf(streambuf_type* obuf = nullptr)
    : basic_syncbuf(obuf, Allocator{});
    basic_syncbuf(streambuf_type* obuf,
                  Allocator const &allocator);
    basic_syncbuf(basic_syncbuf&& other);
    ~basic_syncbuf();

    basic_syncbuf& operator=(basic_syncbuf&& rhs);
    void swap(basic_syncbuf &other);

    bool emit();
    streambuf_type* get_wrapped() const noexcept;
    allocator_type get_allocator() const noexcept;

protected:
    int sync() override;

private:
    streambuf_type *wrapped; // exposition only
};

template <class charT, class traits, class Allocator>
inline void swap(basic_syncbuf<charT, traits, Allocator>& a,
                basic_syncbuf<charT, traits, Allocator>& b);
```

basic_syncbuf constructors [syncstream.syncbuf.ctor]

```
explicit basic_syncbuf(streambuf_type* obuf = nullptr, Allocator const &allocator = Allocator());
```

Effects: Constructs the `basic_syncbuf` object and sets `wrapped` to `obuf` which will be the final destination of characters inserted into the stream.

Remarks: If `obuf == nullptr`, output operations on `*this` will fail. A copy of `allocator` is used to allocate memory for internal buffers.

Throws: Nothing unless constructing a mutex or allocating memory throws.

[*Note:* Subsequent calls to `emit()` might result in member functions being called on the user-provided stream buffer while a lock is held. —*end note*]

Postconditions: `get_wrapped() == obuf` && `get_allocator() == allocator`..

```
basic_syncbuf(basic_syncbuf&& other);
```

Effects: If `other.get_wrapped() == nullptr`, equivalent to `basic_syncbuf(nullptr, other.get_allocator())` otherwise the state necessary to perform `other.emit()` is moved to `*this`.

Postconditions: The value returned by `this->get_wrapped()` is the value returned by `other.get_wrapped()` prior to calling this constructor. Output stored in `other` prior to calling this constructor will be stored in `*this` afterwards. `other.rdbuf()->pbase() == other.rdbuf()->pptr()` and `other.get_wrapped() == nullptr`

[Note: This constructor disassociates `other` from its wrapped stream buffer ensuring destruction of `other` produces no output. —end note]

basic_syncbuf destructor [syncstream.syncbuf.dtor]

`~basic_syncbuf();`

Effects: Calls `this->emit()`.

Throws: Nothing. If an exception is thrown while writing to the wrapped buffer, that exception is caught and ignored.

basic_syncbuf assign and swap [syncstream.syncbuf.assign]

`basic_syncbuf& operator=(basic_syncbuf&& rhs);`

Effects: Calls `this->emit()`. If `rhs.get_wrapped() != nullptr`, the state necessary to perform `rhs.emit()` is moved to `*this`.

Returns: `*this`.

Postconditions: `other.get_wrapped() == nullptr`. If `allocator_traits<Allocator>::propagate_on_container_move_assignment::value == true`, then `this->get_allocator() == other.get_allocator()`; otherwise the allocator is unchanged.

[Note: This assignment operator disassociates `other` from its wrapped stream buffer ensuring destruction of `other` produces no output. —end note]

`void swap(basic_syncbuf& other);`

Preconditions: `this->get_allocator() == other.get_allocator()`.

Effects: Exchanges the state of `*this` and `other`.

`template <class charT, class traits, class Allocator> void swap(basic_syncbuf<charT,traits,Allocator>& a, basic_syncbuf<charT,traits,Allocator>& b);`

Effects: As if by `a.swap(b)`.

basic_syncbuf member functions [syncstream.syncbuf.mfun]

`bool emit();`

Effects: Atomically transfers the contents of the internal buffer to the stream buffer `*wrapped`, so that they appear in the output stream as a contiguous sequence of characters. If and only if a call was made to `this->sync()`, `wrapped->pubsync()` is called.

Returns: `true` if `wrapped != nullptr`, all of the buffered characters were successfully transferred, and the call to `wrapped->pubsync()` (if any) succeeded; otherwise `false`.

Synchronization: All `emit()` calls transferring characters to the same stream buffer object appear to execute in a total order consistent with *happens-before* where each `emit()` call synchronizes-with subsequent `emit()` calls in that total order.

Remarks: May call member functions of `wrapped` while holding a lock uniquely associated with `wrapped`.

`streambuf_type* get_wrapped() const noexcept;`

Returns: wrapped.

allocator_type get_allocator() const noexcept;

Returns: A copy of the allocator set in the constructor or from assignment.

basic_syncbuf overridden virtual functions [syncstream.syncbuf.virtuals]

int sync() override;

Effects: Record the fact that user desires that the wrapped buffer be flushed. The actual flush is delayed until a call to emit().

Returns: 0.

Class template basic_osyncstream [syncstream.osyncstream]

```
template <class charT,  
         class traits,  
         class Allocator>  
class basic_osyncstream  
: public basic_ostream<charT,traits>  
{  
public:  
    using char_type      = charT;  
    using int_type        = typename traits::int_type;  
    using pos_type        = typename traits::pos_type;  
    using off_type        = typename traits::off_type;  
    using traits_type      = traits;  
    using allocator_type   = Allocator;  
    using streambuf_type   = basic_streambuf<charT,traits>;  
    using syncbuf_type     = basic_syncbuf<charT,traits, Allocator>;  
  
    explicit basic_osyncstream(streambuf_type *obuf)  
    :basic_osyncstream(obuf, Allocator{});  
    explicit basic_osyncstream(basic_ostream<charT,traits> &os)  
    :basic_osyncstream(os,Allocator{});  
    basic_osyncstream(streambuf_type *obuf, Allocator const &allocator);  
    basic_osyncstream(basic_ostream<charT,traits> &os, Allocator const &allocator);  
    basic_osyncstream(basic_osyncstream&&);  
    ~basic_osyncstream();  
    basic_osyncstream& operator=(basic_osyncstream&&);  
    void emit();  
    streambuf_type* get_wrapped() const noexcept;  
    syncbuf_type* rdbuf() const noexcept override { return &sb ; }  
private:  
    syncbuf_type sb; // exposition only  
};
```

Allocator shall meet the allocator requirements [allocator.requirements].

[Example: Use a named variable within a block statement for streaming across multiple statements.

```
{  
    osyncstream bout(cout);  
    bout << "Hello, ";  
    bout << "World!";  
    bout << endl; // flush is noted  
    bout << "and more!\n";  
} // characters are transferred and cout is flushed
```

—end example]

[Example: Use a temporary object for streaming within a single statement. cout is not flushed.

```
osyncstream(cout) << "Hello, " << "World!" << '\n';
```

—end example]

osyncstream Constructor [syncstream.osyncstream.ctor]

```
basic_osyncstream(streambuf_type *buf, Allocator const &allocator);
```

Effects: Initializes sb from buf and allocator and initializes the base class with basic_ostream(&sb).

[Note: If wrapped stream buffer pointer refers to a user provided stream buffer then its implementation must be aware that its member functions might be called from emit() while a lock is held. —end note]

Postconditions: get_wrapped() == buf

```
basic_osyncstream(basic_ostream<charT, traits>& os, Allocator const &allocator);
```

Effects: basic_osyncstream(os.rdbuf(),a)

```
basic_osyncstream(basic_osyncstream&& other);
```

Effects: Move constructs from other. This is accomplished by move constructing the base class, and the contained basic_syncbuf sb. Next basic_ostream<charT, traits>::set_rdbuf(&sb) is called to install the basic_syncbuf sb.

Postconditions: The value returned by this->get_wrapped() is the value returned by os.get_wrapped() prior to calling this constructor. nullptr == os.get_wrapped().-->

osyncstream Destructor [syncstream.osyncstream.dtor]

```
~basic_osyncstream();
```

Effects: Calls emit().

osyncstream Assignment [syncstream.osyncstream.assign]

```
basic_osyncstream& operator=(basic_osyncstream&& other);
```

Effects: First, calls this->emit(). Move assigns sb from other.sb. [Note: This disassociates other from its wrapped stream buffer ensuring destruction of other produces no output. —end note]

Postcondition: Primarily, nullptr == other.get_wrapped(). Also, get_wrapped() returns the value previously returned by other.get_wrapped().

osyncstream Member Functions [syncstream.osyncstream.mfun]

```
void emit();
```

Effects: Calls sb.emit(). If this call returns false, calls setstate(ios::badbit).

[Example: A flush on a basic_osyncstream does not flush immediately:

```
{
    _osyncstream bout(cout);
    bout << "Hello," << '\n'; // no flush
    bout.emit(); // characters transferred; cout not flushed
    bout << "World!" << endl; // flush noted; bout not flushed
    bout.emit(); // characters transferred; cout flushed
    bout << "Greetings." << '\n'; // no flush
} // characters transferred; cout not flushed
```

—end example]

[Example: The function `emit()` can be used to catch exceptions from operations on the underlying stream.

```
{
  ostream bout(cout);
  bout << "Hello, " << "World!" << '\n';
  try {
    bout.emit();
  } catch ( ... ) {
    // stuff
  }
}
```

—end example]

`streambuf_type* get_wrapped() const noexcept;`

Returns: `sb.get_wrapped()`

[Example: Obtaining the wrapped stream buffer with `get_wrapped()` allows wrapping it again with an `ostream`. For example,

```
{
  ostream bout1(cout);
  bout1 << "Hello, ";
  {
    ostream bout2(bout1.get_wrapped());
    bout2 << "Goodbye, " << "Planet!" << '\n';
  }
  bout1 << "World!" << '\n';
}
```

produces the *uninterleaved* output

```
Goodbye, Planet!
Hello, World!
```

—end example.]

Implementation

Two example implementations are available on github.com/PeterSommerlad/SC22WG21_Papers/workspace/Test_basic_ostream and https://github.com/PeterSommerlad/SC22WG21_Papers/tree/master/workspace/p0053_basic_ostreambuf.

The latter actually implements almost all of the mechanics in a stream buffer as proposed by Pablo Halpern where the first one is my original implementation with most of the mechanics stuck in the `basic_ostream`.

Revisions

This paper revises [P0053R3 C++ Synchronized Buffered Ostream](#)

- Takes input from Pablo Halpern and re-instantiate the stream buffer that performs the synchronization.
- Split the constructors with a defaulted allocator parameter to one single-argument one being explicit and one non-explicit taking 2 arguments.

This paper revises [P0053R2 C++ Synchronized Buffered Ostream](#)

- Remove the "may construct a mutex" notes.
- Remove "may destroy mutex" notes.
- Clarify `ostream` flush behavior in an example.
- Make minor editorial fixes.

[P0053R2](#) revised [P0053R1 C++ Synchronized Buffered Ostream](#)

- Provide a typedef for the wrapped stream buffer and use it to shorten the specification as suggested by Daniel Krügler.
- Provide move construction and move assignment and specify the moved-from state to be detached from the wrapped stream buffer.
- Rename `get()` to `rdbuf_wrapped()` and provide `noexcept` specification.
- Changed to explicitly rely on wrapping a stream buffer, instead of an ostream object and adjust explanations accordingly.

[P0053R1](#) revised [P0053R0 C++ Synchronized Buffered Ostream](#)

- Add remark to note that exchanging the stream buffer while the stream is wrapped causes undefined behavior and added a note to warn stream buffer implementers about the lock being held in `emit()`. Call `setstate(badbit)` if IO errors occur in `emit()`.
- Replace code references to `basic_streambuf` by the term *stream buffer* introduced in `[stream.buffer]`.
- Provide an example implementation.
- The lock is to be associate to the underlying `basic_streambuf` instead of the `basic_stream`.
- Added an Allocator constructor parameter.
- Moves destructor example to `emit()`.
- Clarifies wording about synchronization and flushing (several times).
- List the new header in corresponding table.
- Provide type aliases in `<iosfwd>`.
- Removed copy constructor in favor of providing `get()`.
- Notify that move construction and assignment is deleted.
- Moved class `noteflush_streambuf` into an implementation note.
- Add a design subsection that states that a header test is a sufficient feature test.

[P0053R0](#) revised [N4187 C++ Ostream Buffers](#)

- Updated introduction with recent history.
- Rename `ostream_buffer` to `osyncstream` to reflect its appearance is more like a stream than like a buffer.
- Add an example of using `osyncstream` as a temporary object.
- Add an example of a `osyncstream` constructed with another `osyncstream`.
- Clarify the behavior of nested `osyncstream` executions.
- Clarify the behavior of exceptions occurring with the `osyncstream` destructor.
- Clarify the deferral of flush from the `osyncstream`'s `streambuf` to the final `basic_ostream`.
- Limit the number of references to `noteflush_stringbuf` in anticipation of the committee removing it from the specification.
- Rename `noteflush_stringbuf` to `noteflush_streambuf` to hide possible implementation details.
- Change the base class of `noteflush_streambuf` from `basic_stringbuf` to `basic_streambuf`.

[N4187](#) revised [N4069 C++ Ostream Buffers](#)

- Added note to sync as suggested by BSI via email.

[N4069](#) revised [N3978 C++ Ostream Buffers](#)

- Added a Design section.
- Clarify the reference capturing behavior of the `ostream_buffer` constructors.
- Added `noexcept` and `const` as appropriate to members.

- Added note on throwing wrapped streams.
- Change the `noteflush_stringbuf` public member variable `needsflush` to a public member query function `flushed`.
- Removed the public member function `noteflush_stringbuf::clear`.
- Minor synopsis formatting changes.
- Incorporated feedback from SG1 and Dietmar Kühl in specific in Rapperswil.

[N3978](#) revised [N3892 C++ Ostream Buffers](#)

- Flush the ostream if and only if the `ostream_buffer` was flushed.
- Add the `clear_for_reuse` function.
- Change the design from inheriting from `basic_ostream` to using a `noteflush_stringbuf`, which is a slightly modified `basic_stringbuf`. The modification is to note the flush rather than act upon it.

[N3892](#) revised [N3750 C++ Ostream Buffers](#)

- Change name to `basic_ostream_buffer` and add the usual typedefs.
- Change interface to inherit from `basic_ostringstream` rather than provide access to a member of that type.
- Add a Revisions section.

References

[PSL]

The Open Group Base Specifications Issue 6, IEEE Std 1003.1, 2004 Edition, functions, flockfile,
<http://pubs.opengroup.org/onlinepubs/009695399/functions/flockfile.html>